

# SQL Companion for SSMS

Cross-table column search, a foreign-key relationships explorer with click-to-insert JOINS, and an auto-declare-variables command — all inside SQL Server Management Studio 20.

User Guide · Publisher: LinkDev · Applies to SSMS 20 (Visual Studio 2017 / 15.0 isolated shell)

## Contents

1. Overview
2. Requirements
3. Installation
4. Getting started
5. Feature 1 — Column Search
6. Feature 2 — Relationships
7. Feature 3 — Declare Variables
8. Tips & troubleshooting
9. Sample test database

## 1 • Overview

SQL Companion adds three productivity tools to SSMS 20, reachable from the **Tools ▸ SQL Companion** submenu, the **SQL Companion** toolbar, and the query-editor right-click menu:

| Tool                     | What it does                                                                                       |
|--------------------------|----------------------------------------------------------------------------------------------------|
| <b>Column Search</b>     | Find every table/view column whose name matches your text — across a database or the whole server. |
| <b>Relationships</b>     | Browse a table's foreign-key relationships (both directions) and insert ready-made JOINS.          |
| <b>Declare Variables</b> | Scan the current batch and add <code>DECLARE</code> statements for variables used without one.     |

Both tool windows read the **active query window's connection**, so connect and open a query window first.

## 2 • Requirements

- **SQL Server Management Studio 20** (built on the Visual Studio 2017 / 15.0 isolated shell).
- **.NET Framework 4.7.2** or later (installed with SSMS 20).
- Administrator rights on the machine *for installation only* (the extension is copied under SSMS's program folder).

## 3 • Installation

**Do not double-click the .vsix** . SSMS 20's standalone VSIX installer cannot install this add-in (it reports *"not installable on any currently installed products"*). SQL Companion must be deployed into SSMS's machine-wide extensions folder and registered with a one-time menu merge. The included `deploy.ps1` does this for you.

## Install with deploy.ps1 (recommended)

- 1 Close SSMS.
- 2 Open **Windows PowerShell as Administrator**.
- 3 Run it from the project folder:

```
cd C:\path\to\SQLCompanion
.\deploy.ps1                # build + deploy + register
.\deploy.ps1 -Configuration Release
.\deploy.ps1 -Uninstall      # remove it
```

- 4 Start SSMS 20. The **Tools** ▶ **SQL Companion** submenu appears.

The script builds the project, copies `extension.vsixmanifest` , `SQLCompanion.dll` and `SQLCompanion.pkgdef` into `...\Microsoft SQL Server Management Studio 20\Common7\IDE\Extensions\SQLCompanion\` , and runs `ssms.exe /setup` to merge the menus. It self-elevates via UAC if needed.

## What the installer produces

After a successful install you will see, under **Tools** ▶ **SQL Companion**:

[Column Search](#) [Relationships](#) [Declare Variables](#)

## 4 • Getting started

- 1 Open SSMS and **connect to a server**.
- 2 Open a **new query window** (`Ctrl + N`) — this is the connection the tools read.
- 3 Open a tool from **Tools** ▶ **SQL Companion** (or the **SQL Companion** toolbar).

Each tool window shows the current connection at the top ( `Connected: server / database` ). If it says *"No active SQL connection"*, click into a connected query window and press **Refresh**.

# 5 · Feature 1 — Column Search

Find columns by name across many tables and views — ideal when you know a column exists but not where.

## How to use

- 1 Open **Tools** ▸ **SQL Companion** ▸ **Column Search**.
- 2 Type a full or partial column name (matching is **case-insensitive** and **partial**) and press `Enter` or **Search**.
- 3 Refine with the **Scope**, **Schema**, and **Recent** controls.

## Controls

| Control               | Purpose                                                         |
|-----------------------|-----------------------------------------------------------------|
| Scope                 | <b>Current database</b> or <b>All databases (server)</b> .      |
| Schema                | Show only results from one schema (or all).                     |
| Recent                | Re-run one of your recent searches.                             |
| Export CSV / Copy All | Save results to a CSV file, or copy them as tab-separated text. |
| Refresh               | Re-read the active server/database from SSMS.                   |

## Results grid

Columns: **Database**, **Schema**, **Table/View**, **Type**, **Column**, **Data Type**, **Nullable**, **Key**. The **Key** badge shows `PK`, `FK`, or `PK,FK`. Click a header to sort.

## Row actions

- **Double-click** a row to insert the fully-qualified `[schema].[table]` at the editor cursor.
- **Right-click** a row for: Insert table name, Insert column name, Generate `SELECT`, Copy column name, Copy table name.

## 6 · Feature 2 — Relationships

Explore a table's foreign keys in both directions and drop ready-made JOINS into your query.

### How to use

- 1 Open **Tools** ▶ **SQL Companion** ▶ **Relationships**.
- 2 Pick a **Database** from the dropdown — the **Table** dropdown then fills with that database's tables.
- 3 Pick a **Table** (or type `schema.table` / click **Use active table**) to load its relationships.
- 4 Choose **Join** type ( `INNER` / `LEFT` ), then **double-click** a row to insert the JOIN.

**The Table dropdown depends on the Database dropdown.** Changing the database repopulates the table list. Use **Refresh** to reload both lists (e.g., after you connect).

### Results grid

| Column                 | Meaning                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------|
| Direction              | → references (this table points to another) or ← referenced by (another table points here). |
| Related schema / table | The table on the other side of the relationship.                                            |
| Joining columns        | The column pairs that make up the foreign key.                                              |
| FK name                | The foreign-key constraint name.                                                            |

### Inserting a JOIN

Double-click a row (or right-click ▶ **Insert JOIN at cursor**) to insert an `INNER` / `LEFT JOIN` built from the foreign-key columns. The base table is assumed to be aliased `t` in your `FROM` clause — rename after pasting if needed. Composite (multi-column) keys are fully supported.

## 7 · Feature 3 — Declare Variables

---

Turn a snippet full of `@variables` into runnable T-SQL without hand-writing every `DECLARE`.

### How to use

- 1 In a query window, write or paste T-SQL that **uses** variables (e.g. `SET @id = 1; ... WHERE x = @id`).
- 2 Run **Declare Variables** from the **Tools** ▶ **SQL Companion** submenu, the toolbar, or the editor **right-click** menu.
- 3 Missing `DECLARE` statements are inserted at the top of the batch, with a type inferred from how each variable is used.

Only variables that are **used without a DECLARE** are added; variables you already declared are left untouched. If nothing is missing, a status message says so.

## 8 · Tips & troubleshooting

| Symptom                                                               | Fix                                                                                                                   |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| The <b>SQL Companion</b> submenu is missing from Tools                | The menu merge didn't run. Re-run <code>deploy.ps1</code> (it runs <code>ssms.exe /setup</code> ), then restart SSMS. |
| Dropdowns / results are empty; header says "No active SQL connection" | Open a query window that's connected to a server, then click <b>Refresh</b> in the tool window.                       |
| Relationships Table dropdown is empty                                 | Pick a <b>Database</b> first, or click <b>Refresh</b> . Views aren't listed here (foreign keys live on tables).       |
| Inserted JOIN uses the wrong alias                                    | The base table is assumed to be aliased <code>t</code> ; rename it in the pasted clause to match your query.          |
| Double-clicking the <code>.vsix</code> fails to install               | Expected — use <code>deploy.ps1</code> instead (see Installation).                                                    |

### Uninstall

```
.\deploy.ps1 -Uninstall
```

Removes the extension folder and re-runs the menu merge. Restart SSMS afterwards.

## 9 · Sample test database

The project ships `TestDatabase.sql`, which builds two databases full of the shapes these tools exercise:

- **Repeated column names** across many tables/views and across two databases — great for Column Search (try `name`, `id`, `date`, and the **All databases** scope).
- **Foreign keys of every kind** — self-referencing, cross-schema, and composite (multi-column) — for the Relationships panel. Try loading `sales.Order`, `inventory.Stock`, or `hr.Employee`.
- A ready **Declare Variables** snippet at the bottom of the file.

Run the whole script in SSMS (it's safe to re-run — it drops and recreates the databases).